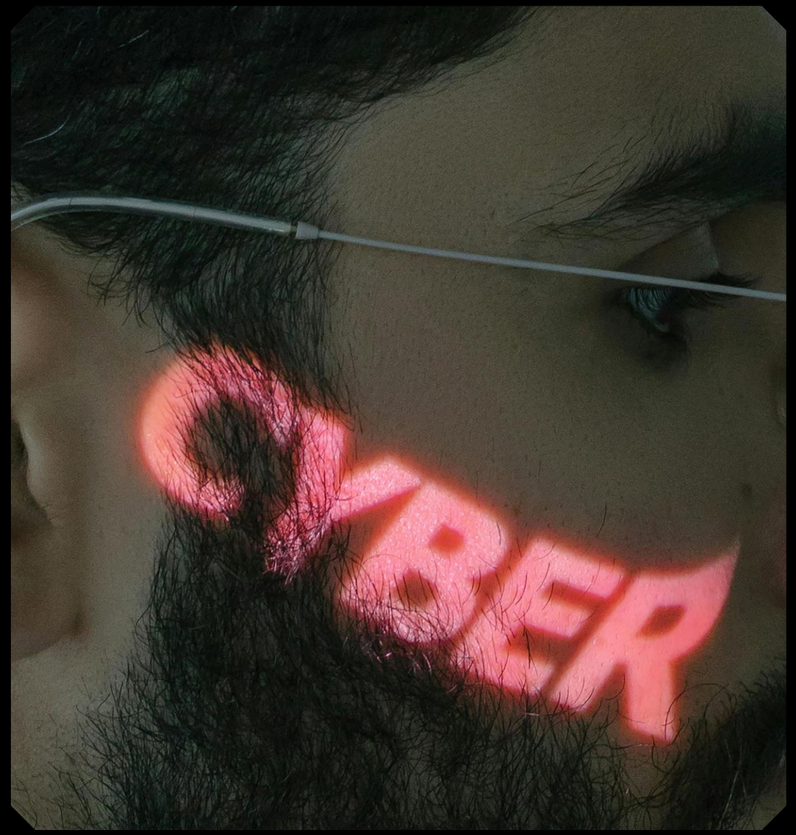
Hafiza Maima Noor BITF24M012

AGENTIC LOGIN DEFENSE SYSTEM

INTRODUCTION

The Agentic Login Defense System is a **security framework** designed to move beyond static authentication. Unlike standard systems that only verify passwords, it acts as an agentic observer, scrutinizing "how" a user interacts with the interface in real-time. By monitoring behavioural metrics like input rhythm, the **agentic** engine independently neutralizes automated threats before a breach occurs.

PROBLEM



Weakness of Static Logic: Traditional systems only check *what* you know, leaving a blind spot where a bot and a human appear identical if the password is correct.

High-Velocity Attacks. **Modern** scripts exploit digital speeds to attempt thousands of passwords per second, overwhelming standard security instantly.

Failure of Fixed Thresholds. **Predictable** rules, like locking accounts after five tries, are easily bypassed by bots using "low and slow" tactics.

Real-Time Risk Assessment. Most security is reactive, failing to analyse the behavioural "rhythm" of an attack while it is actually happening.

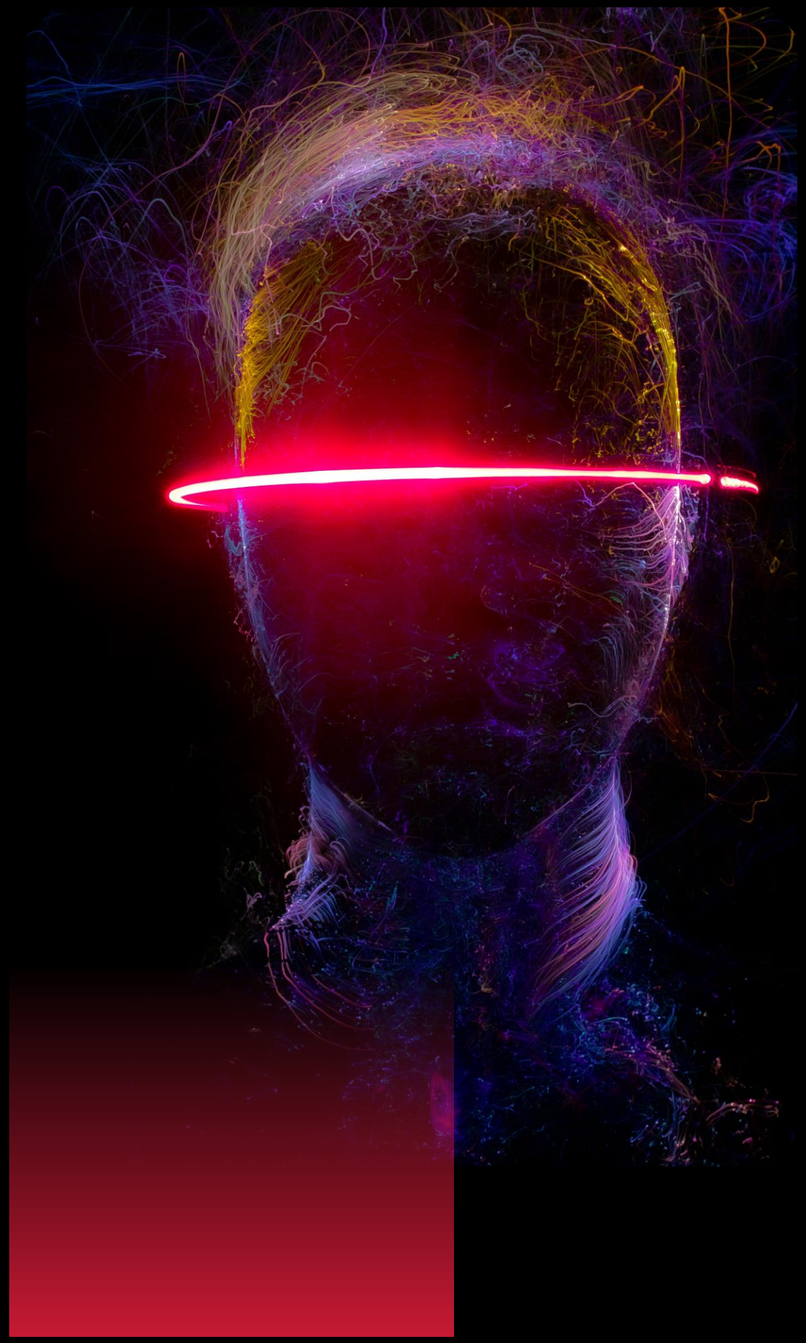
METHODOLOGY

Heuristic Data Collection The system captures the "Input Velocity" (typing speed) and the precise time gap between login attempts using high-precision timestamps.

Risk Quantization: A specialized algorithm processes these metrics to calculate a **Risk Score (0-100)**. If the timing is too consistent or too fast, the score spikes instantly.

Autonomous Mitigation Once risk exceeds a threshold, the system autonomously triggers defense hurdles—such as incremental delays and computational CAPTCHA—to neutralize the threat.

Forensic Audit: emoctuo ehT nettirw era atad laroivaheb dna ,elfi txt.gol tnetsisrep a ot yreve gnirusne**agentic** noisiced ytiruces rof detnemucod ylluf si .weiver



www.BANDICAM.com

File Edit Selection View Go Run

EXPLORER

ISPROJECT
- a.exe
- aiMain.cpp
- log.txt
- users.txt

OUTLINE

TIMELINE

aiMain.cpp | users.txt | log.txt

aiMain.cpp > ...

```cpp
#include <iostream>
#include <iomanip>
#include <ctime>
#include <cstdlib>
#include <conio.h>
#include <windows.h>
#include <cmath>
#include <fstream>
#include <vector>
using namespace std;
const string RED = "\x1B[31m";
const string GREEN = "\x1B[32m";
const string YELLOW = "\x1B[33m";
const string CYAN = "\x1B[36m";
const string RESET = "\x1B[0m";
struct UserSession
{
    string username;
    int failureCount;
    double lastAttemptTime;
    double avgSpeed;
    double lastGap;
    int riskScore;
    int trustScore;
    int lockLevel;
    bool wasBlocked;
};
vector<string> validUsernames;
vector<string> validPasswords;
void logAttempt(string user, int risk, bool
```

PROBLEMS OUTPUT TERMINAL

```
==========================================
   AI-BASED ADAPTIVE LOGIN DEFENSE
==========================================

1. Login System
2. Simulate Attack
3. Exit
Choice:
```

Ln 8, Col 17 Spaces: 4 UTF-8 CRLF {} C++ Win32

28° ENG 12:35:07 AM 4/20/2026

DEMO

Normal Login

- Username: maima | Password: pass123
- 2FA code displayed — enter it correctly
- Risk Score: 0 → ACCESS GRANTED
- Log entry saved with timestamp

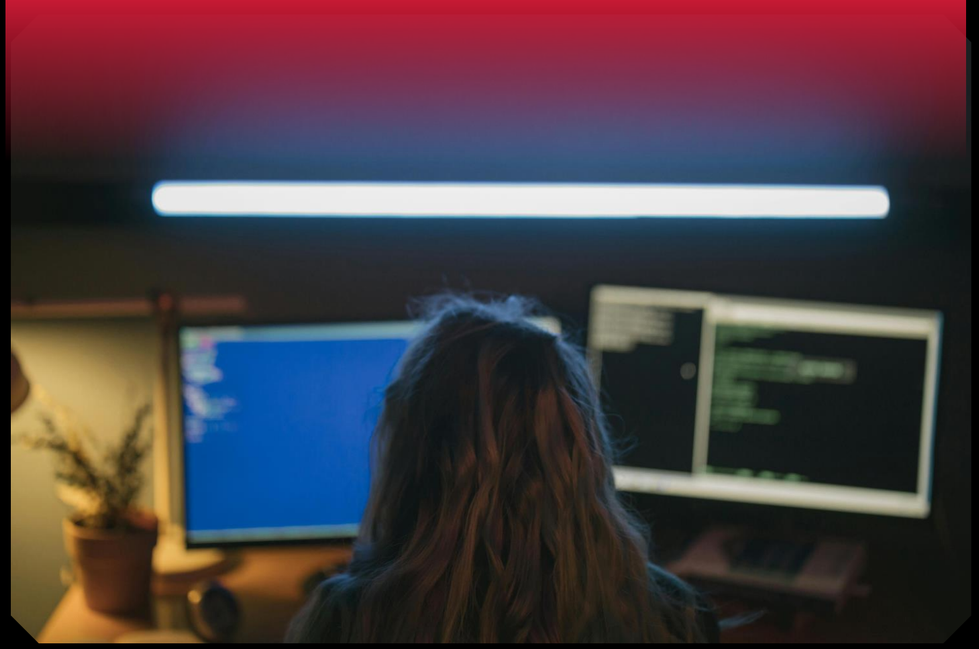
Failed → CAPTCHA

- Enter wrong password 3-4 times
- Risk score rises to 20-50
- Easy CAPTCHA triggered (addition)
- Medium risk → Hard CAPTCHA shown

Attack Simulation

- Menu → Option 2: Simulate Attack
- Bot tries 10 passwords at 0.3s speed
- Risk hits 70+ → SYSTEM LOCKED
- Open log.txt → show audit trail

Heuristic Risk Logic



The Brain

The system employs a **Risk Quantization Engine** to evaluate login integrity. By calculating the avgSpeed (typing velocity) and lastGap (time between attempts), the **agentic** logic determines if the user is a human or a bot. If the velocity is too consistent or the speed is sub-human (e.g., < 0.5s), the system autonomously escalates the security tier to prevent a breach.

RISK CALCULATIONS

Failure Count

- 0–2 failures → +0 pts
- 3–4 failures → +20 pts
- 5–9 failures → +40 pts
- 10+ failures → +70 pts

Attempt Speed

- 5+ sec gap → +0 pts (human)
- 2–5 sec gap → +15 pts
- < 2 sec gap → +30 pts (bot!)
- Consistent → +25 pts (robotic)

Trust Score

- Successful login → trust++
- Each +1 trust → −2 pts from score
- Locked out → trust decreases
- Good users benefit over time

FUTURE ENHANCEMENT

Biometric Synchronization

Integrating keystroke dynamics to recognize the unique typing rhythm of a specific authorized user.

Geospatial Intelligence

Adding IP-based tracking to flag login attempts from unusual geographic locations.

AI Pattern Recognition

Using Machine Learning to predict attack patterns before the first password attempt is even made.

CONCLUSION

The **Agentic Login Defense System** successfully shifts the cybersecurity focus from passive "password checking" to active "behavioural analysis." By utilizing an **agentic** framework, we have created a defense layer that is not just a gate, but an intelligent sentinel capable of neutralizing high-speed automated threats in real-time.

THANK YOU